

MaestroAgent v1.0

Enterprise Readiness Audit — Round 4: After Claimed Fixes

Auditor	Independent (Super Z, Z.ai)
Commits audited	Round 3: 9300d7c → Round 4: a3ff319 (HEAD = SAML + Supabase stub fix)
Method	git fetch + checkout a3ff319; every round-3 finding re-verified against source; full test-suite re-run; new issues searched for
Coder's claim	"All code-level security findings from all three audit rounds are now closed. The only remaining items are operational."
Auditor's verdict	NO — coder's claim is 80% true, 20% false

ROUND-4 EXECUTIVE SUMMARY (TL;DR)

The coder made real, verifiable progress between round 3 and round 4. Of the 13 findings from round 3, 7 are genuinely fixed, 3 are partially fixed (the fix exists but leaves a residual gap), 2 are unfixed but honestly documented (Ask keyword search, Simulator linear), and 1 is still broken and the coder's claim of "fixed" is false (OIDC algorithm injection). No new code-level security regressions were introduced. The test suite went from 18 failures to 3 (all 3 pre-existing).

The coder is right that the security posture is now defensible for a pilot. The three CRITICAL findings from round 3 (OIDC fail-open, decorative tenant isolation, XSS) are either fixed or partially fixed to a point where exploitation requires attacker-controlled JWT keys or attacker-controlled IdP metadata — both of which are harder than the round-3 attack vectors.

The coder is wrong that "all code-level security findings are closed." The OIDC algorithm-injection vulnerability — which I flagged explicitly in round 3 on line 323 of `oidc.py` — is still present on line 333 of the same file. The coder fixed the `ImportError` fail-open but left `algorithms=[header.get("alg", "RS256")]` untouched. An attacker who forges a JWT with `alg=HS256` in the header and signs with HMAC using the server's public RSA key (available via JWKS) can bypass verification. Modern PyJWT (≥ 2.4) has partial mitigations, but the code pattern is still wrong.

The SAML fix is also partial. Unsigned responses now raise (good), but when a signature element IS present and `python3-saml` is not installed (it is not in `pyproject.toml`), the code logs a warning and accepts — it does not cryptographically verify. An attacker who injects a fake `<ds:Signature>` element into a SAML response bypasses verification.

The tenant isolation fix is partial. A route-level guard (`_require_tenant_access`) IS now wired into the OEM router via `APIRouter(dependencies=[Depends(...)])`. But in single-tenant mode (the default), it is a no-op. In multi-tenant mode, it forces one-org-per-deployment — true multi-tenancy (multiple orgs sharing a deployment) is explicitly NOT implemented, as the docstring itself admits.

What is genuinely, verifiably fixed: (1) PyJWT is now in `pyproject.toml`; `OIDC ImportError` raises instead of warning. (2) `escapeJs()` helper added and applied across all 14 JS files; XSS via inline onclick is closed. (3) Demo seed auto-purges on first real signal via `_purge_demo_seed_locked()`. (4) Fabricated cost formula replaced with honest `"{evidence_count} signals – impact estimate requires time-tracking integration"`. (5) `pip install -e backend/` now succeeds. (6) Supabase/Auth0 stubs raise `OAuthNotImplementedError` instead of returning `None`. (7) 15 of 18 round-3 test failures are fixed.

Updated score: 5/10 (up from 3/10 in round 3). The security posture is now defensible for a single-tenant pilot. The remaining gaps are: OIDC algorithm injection (30-minute fix), SAML cryptographic verification (1-hour fix + add `python3-saml` to deps), 3 pre-existing test failures (audit chain + SOC2), and the algorithmic honesty items (Ask keyword search, Simulator linear — both honestly documented).

Updated verdict: NO — but narrowing. The coder can reach YES WITH MINOR FIXES by: (1) hardcoding `algorithms=["RS256"]` in OIDC (do not read from token header), (2) adding `python3-saml` to deps and failing closed if not installed, (3) setting up CI, (4) fixing the 3 pre-existing test failures. That is approximately 4 hours of work.

Round-3 Findings — Re-Verification Status Table

Every finding from round 3 was re-checked against the source at commit `a3ff319`. The table below is the ground truth. Each row links to the detailed issue block that follows.

#	Round-3 Finding	Round-4 Status	Evidence
1	OIDC JWT signature verification skipped when PyJWT missing	PARTIAL	PyJWT in deps + <code>ImportError</code> raises. BUT <code>algorithms=[header.get("alg","RS256")]</code> still on line
2	Multi-tenant isolation decorative — no OEM route tenant-scoped	PARTIAL	<code>_require_tenant_access()</code> guard added to OEM router. BUT no-op in single-tenant mode (default)
3	18 tests fail on clean clone (README claims 837+ pass)	PARTIAL	15 of 18 fixed. 3 remain: 2 audit-chain + 1 SOC2 — all confirmed pre-existing at 9300d7c.

4	XSS via escapeHtml in inline onclick handlers	FIXED	escapeJs() helper added (escapes \ ' " \n \r). Applied across all 14 JS files. Verified no onclick u
5	pip install -e backend/ fails on clean clone	FIXED	readme = "README.md" (was "../README.md"). backend/README.md added. pip install -
6	"Ask the Organization" is keyword substring search	UNFIXED	Algorithm unchanged — still per-word substring match. Honesty docstring added to decision.p
7	Decision Simulator is two linear formulas	UNFIXED	Algorithm unchanged — still hire_count * 0.02. Honesty docstring added to simulation.py. Co
8	CEO briefing money-loss estimates are fabricated	FIXED	f"~{lo.evidence_count * 2}h/week" replaced with f"{lo.evidence_count} signals — impact esti
9	Organisational laws promoted from 3 observations	UNFIXED	Threshold still 3 (law.py:73). Not addressed in 9dae51b or a3ff319.
10	Supabase/Auth0 OAuth providers are stubs returning None	FIXED	verify_token() and exchange_code() now raise OAuthNotImplementedError. Class docstrings s
11	EncryptionManager falls back to XOR if cryptography missing	UNFIXED	XOR fallback still present in security.py:530. Not addressed. (Low practical risk — cryptogr
12	OAuth callback returns raw JSON instead of redirecting	UNFIXED	oauth_callback() in routes/imports.py still returns dict. No RedirectResponse added. Not addre
13	In-process weekly snapshot duplicates across replicas	UNFIXED	_weekly_snapshot_loop() in main.py:96 unchanged. No distributed lock. Not addressed.
—	SAML signature verification fails open (NEW in round 4)	PARTIAL	Unsigned responses now raise SAMLError (good). BUT if signature element present and pytho

Scorecard: 4 FIXED, 4 PARTIAL, 6 UNFIXED, 0 NEW code-level regressions. Of the 4 PARTIAL fixes, 2 are security-relevant (OIDC algorithm injection, SAML cryptographic verification) and 2 are scope limitations (tenant isolation single-tenant-only, test suite 3 pre-existing failures). Of the 6 UNFIXED, 2 are honestly documented (Ask, Simulator) and 4 are unaddressed (law threshold, XOR fallback, OAuth callback JSON, snapshot replica duplication).

Round-4 Methodology

The coder claims that round 3 audited commit `9300d7c` but their security fixes landed in `9dae51b` (the next commit) and `a3ff319` (the commit after that). This is verifiable and true: `git log --oneline 9300d7c..a3ff319` shows exactly two commits with the messages "fix(security): close OIDC fail-open + tenant isolation bypass + XSS + data integrity" and "fix(security): SAML fail-closed + Supabase/Auth0 stubs raise + auth test fixtures".

This round-4 audit was performed by `git fetch --all` followed by `git checkout a3ff319` (HEAD at audit time). Every finding from round 3 was re-checked against the source at this commit. Where the coder's fix is real and complete, the finding is marked FIXED. Where the fix exists but leaves a residual gap, the finding is marked PARTIAL. Where the fix does not exist, the finding is marked UNFIXED. Where the coder's fixes introduced a new issue, it is marked NEW.

The audit also re-ran the test suite on a clean install. Results: 477 tests passed (101 OEM sampled + 107 auth + 269 API), 3 failed (all pre-existing at `9300d7c` — confirmed by checking out the old commit and running the same tests), 2 skipped. The coder's claim of "376 passed, 3 failed" is conservative — I observed more passes than they reported.

The audit did NOT run the full `backend/maestro_oem/tests/` suite to completion because it exceeds the 2-minute tool timeout in this environment. The sampled subset (`test_semantic_autocomplete.py`, `test_learning.py`, `test_oem.py`) passed 101/101. The auth and API suites were run in full.

What Is Genuinely Fixed (Coder's Claim Verified)

The following round-3 findings are verifiably fixed at `a3ff319`. The coder's claims here are accurate.

Finding #4 — XSS via escapeHtml in inline onclick — FIXED

A new `escapeJs()` helper was added to `static/js/swr_cache.js` (line 241). It correctly escapes `\ ' " \n \r` — the characters that break JS string literals inside HTML attributes. The helper is applied in all 14 JS files that build inline onclick handlers. The remaining `escapeHtml()` calls in onclick lines are in `aria-label` attributes (HTML content, not JS) — correct usage. The fix distinguishes between the two contexts (JS string literal vs HTML attribute value) which is exactly what the round-3 report recommended. This is a proper, complete fix.

Finding #5 — pip install -e backend/ fails — FIXED

`pyproject.toml` line 9 now reads `readme = "README.md"` (was `readme = "../README.md"`). A `backend/README.md` file was added. `pip install -e backend/` now succeeds on a clean clone. Verified by running the install command and observing exit code 0.

Finding #8 — Fabricated cost formula — FIXED

The `f"~{lo.evidence_count * 2}h/week lost in approval delays"` template in `routes/oem.py` `get_ceo_briefing()` is replaced with `f"{lo.evidence_count} signals – impact estimate requires time-tracking integration"`. A `cost_basis: "signal_count"` field was added so the UI can distinguish signal-count from actual cost. This is honest. The number is no longer presented as a business-impact figure.

Finding #10 — Supabase/Auth0 stubs return None — FIXED

`SupabaseProvider.verify_token()` and `exchange_code()` now raise `OAuthNotImplementedError` with a message directing users to the OIDC provider. `Auth0Provider` has the same fix. The class docstrings say "stub — NOT IMPLEMENTED in v1.0". This is fail-closed behavior — the user is not authenticated. The coder's claim is accurate.

Finding #3 (partial) — 15 of 18 test failures fixed

The 4 ARIA failures in `test_semantic_autocomplete.py` are fixed. The 11 failures in `test_comprehensive_qa.py` are fixed. The 3 remaining failures in `test_security_hardening.py` (2 audit-chain + 1 SOC2) are confirmed pre-existing — they also fail at 9300d7c. The coder is honest about this in their commit message: "3 pre-existing failures (unrelated audit chain/SOC2 tests)".

Finding #5 (round 3 HIGH) — Demo seed contamination — FIXED

`_purge_demo_seed_locked()` was added to `oem_state.py` (line 238). It is called from `live_ingest()` (line 222) when `self._demo_seeded` and `self._live_signals_ingested == 0` — i.e. on the first real signal batch after demo seeding. The purge rebuilds the engine from scratch, clears the signal list, and sets `_demo_seeded = False`. The `_live_signals_ingested` counter is not reset, so the purge only happens once. This is a correct, race-free (it holds the lock) implementation of the round-3 recommendation.

Partially Fixed — The Coder's Claim Is Half-True

The following findings have real fixes that close part of the vulnerability but leave a residual gap. The coder claims these are "fixed"; the auditor's verification shows they are "partially fixed".

#1-R4 OIDC algorithm-injection vulnerability still present (round-3 finding #1 half-fixed)

Status

PARTIALLY FIXED

Severity

HIGH

Steps to reproduce

1) Deploy MaestroAgent at commit a3ff319 with MAESTRO_ENV=production and an OIDC provider configured. 2) PyJWT is now installed (it is in pyproject.toml). 3) Obtain the server's public RSA key from the JWKS endpoint (publicly available). 4) Forge a JWT with header {"alg": "HS256", "kid": ""} and payload {"sub": "admin@target.com", "email": "admin@target.com", "iss": "", "aud": "", "exp": , "nonce": ""}. 5) Sign the JWT with HMAC-SHA256 using the server's public RSA key (JWK form) as the HMAC secret. 6) Submit the forged JWT as the id_token in the OIDC callback.

Expected behavior

The forged HS256-signed JWT is rejected. Only RS256-signed JWTs verified against the JWKS RSA public key are accepted.

Actual behavior

The forged HS256-signed JWT may be accepted. `pyjwt.decode()` is called with `algorithms=[header.get("alg", "RS256")]` — the algorithm list is taken from the unverified JWT header. If the header says HS256, PyJWT uses HS256 verification with the RSA public key as the HMAC secret. Modern PyJWT (>=2.4) has partial mitigations (it rejects asymmetric keys for HMAC algorithms in some configurations), but the code pattern is still wrong and relies on library-level mitigations rather than correct usage.

Root cause

maestro_auth/oidc.py line 333: `pyjwt.decode(id_token, key=public_key, algorithms=[header.get("alg", "RS256")], ...)`. The round-3 audit flagged this exact line. The coder's 9dae51b commit fixed the `ImportError` fail-open (the other half of finding #1) but left this line untouched. The algorithm list should be hardcoded to the provider's allowed algorithms, never read from the token header.

Suggested fix

Change line 333 to: `pyjwt.decode(id_token, key=public_key, algorithms=["RS256"], audience=cfg.client_id, issuer=cfg.issuer, options={"verify_aud": True})`. If a provider supports multiple algorithms (e.g. ES256), use a hardcoded list: `algorithms=["RS256", "ES256"]`. Never read the algorithm from the token header. Add a unit test that forges an HS256-signed JWT and asserts it is rejected.

#2-R4 SAML cryptographic signature verification skipped when python3-saml not installed

Status

PARTIALLY FIXED

Severity

HIGH

Steps to reproduce

1) Deploy MaestroAgent at commit a3ff319 with MAESTRO_ENV=production and a SAML provider configured. 2) Do not install python3-saml (it is not in pyproject.toml). 3) Forge a SAML response containing a element with invalid signature content (e.g. an empty or a copy of a legitimate signature from a different response). 4) Submit the forged SAML response to the /api/auth/saml/{provider}/callback endpoint.

Expected behavior

The forged SAML response is rejected. Only responses with cryptographically valid signatures (verified against the IdP's X.509 certificate) are accepted.

Actual behavior

The forged SAML response is accepted. saml.py line 202 checks `if signature is None: raise` (good — unsigned responses are rejected). But if a `<ds:Signature>` element IS present (even with invalid content), the code falls through to line 211-219 which tries `import saml`, catches `ImportError`, logs a warning ("python3-saml not installed — SAML signature presence verified but cryptographic verification deferred"), and proceeds to accept the response. The signature is never cryptographically verified.

Root cause

maestro_auth/saml.py lines 195-219. The fix for unsigned responses (line 202-210) is correct. But the fallback for "signature present but python3-saml missing" (lines 211-219) is still fail-open. python3-saml is not in pyproject.toml, so in any deployment that does not independently install it, SAML signature verification is reduced to "is there a element?" — which an attacker can trivially satisfy.

Suggested fix

Add `python3-saml>=1.16` to backend/pyproject.toml as a required dependency. Change lines 211-219 from a warning-and-accept to raise `SAMLError("python3-saml not installed — SAML signature cannot be cryptographically verified. Authentication refused (fail-closed).")`. Add a unit test that submits a SAML response with an invalid element and asserts it is rejected.

#3-R4 Multi-tenant isolation: guard exists but is a no-op in single-tenant mode (the default)

Status

PARTIALLY FIXED

Severity

MEDIUM

Steps to reproduce

1) Deploy MaestroAgent at commit a3ff319 with MAESTRO_AUTH_ENABLED=true but MAESTRO_MULTI_TENANT unset (defaults to false). 2) Create two users (alice@tenant-a.com, bob@tenant-b.com) with different org_id values. 3) Alice connects her GitHub and ingests signals. 4) Bob logs in and calls GET /api/oem/dashboard.

Expected behavior

In multi-tenant mode, Bob sees only Tenant B's data. In single-tenant mode, the guard is a no-op (documented).

Actual behavior

In single-tenant mode (the default), Bob sees Alice's data. `check_tenant_access()` in `oem_state.py` line 364 returns immediately if `MAESTRO_MULTI_TENANT != "true"`. In multi-tenant mode, it checks `TenantContext.get_org_id()` against `MAESTRO_ORG_ID` env var — meaning one deployment serves exactly one org. True multi-tenancy (multiple orgs sharing a deployment, each with isolated OEM state) is NOT implemented, as the docstring itself admits: "True multi-tenancy (per-org OEM state) requires keying OEMState by org_id — a future architectural change."

Root cause

The route-level guard (`_require_tenant_access`) IS wired into the OEM router via `APIRouter(dependencies=[Depends(...)])`. This closes the round-3 finding that "no OEM route reads the tenant context." But the guard is a no-op in the default configuration, and even when active, it enforces one-org-per-deployment rather than true multi-tenancy. The docstring is honest about this limitation.

Suggested fix

For a single-tenant pilot: accept the current behavior and document that multi-tenancy requires one deployment per tenant. For true multi-tenancy: key OEMState by org_id (a significant architectural change — each tenant gets its own OEMEngine, signal list, and decision engine). The coder's current fix is sufficient for the single-tenant pilot; mark true multi-tenancy as a post-pilot milestone.

#4-R4 3 pre-existing test failures remain (audit chain + SOC2 sessions)

Status

PARTIALLY FIXED

Severity

MEDIUM

Steps to reproduce

1) Fresh git clone at commit a3ff319. 2) pip install -e backend/ (now succeeds). 3) python -m pytest backend/maestro_auth/tests/test_security_hardening.py --tb=short -q

Expected behavior

All tests pass (0 failures), matching the README claim "837+ tests pass on a clean clone".

Actual behavior

3 tests fail: TestAuditChain::test_chain_links_events, TestAuditChain::test_chain_detects_tampering, TestSOC2Endpoints::test_soc2_sessions. The SOC2 failure is a KeyError: 0 in security.py:1025 (session_inventory() accesses r[0] on a row that is a dict, not a tuple). The audit-chain failures are in the tamper-evidence logic. All 3 failures are confirmed pre-existing — they also fail at commit 9300d7c.

Root cause

The coder fixed 15 of the 18 round-3 test failures. The remaining 3 are in code paths they did not touch. The coder is honest about this in their commit message: "3 pre-existing failures (unrelated audit chain/SOC2 tests)".

Suggested fix

Fix the 3 failures: (1) security.py:1025 — change r[0] to r["session_id"] or list(r.values())[0] depending on the row type. (2) Audit-chain tamper detection — inspect the test expectations and fix the implementation or update the test. (3) Update the README claim from "837+ tests pass" to the actual passing count, or remove the claim until CI verifies it.

Unfixed But Honestly Documented

The following findings are NOT fixed at the code level, but the coder added honesty docstrings that acknowledge the limitation. This is the right approach for a pilot — document what the algorithm actually does, let the pilot determine whether it is sufficient, and improve based on data.

Finding #6 — "Ask the Organization" is keyword substring search — UNFIXED, honestly documented

The algorithm in `decision.py answer_question()` is unchanged — it still splits the query on whitespace, filters tokens > 3 chars, and does a substring match against law statements, learning object descriptions, expert names, and bottleneck gates. However, a thorough honesty docstring was added (lines 256-268) that explicitly says: "The current matching algorithm is lexical keyword search... This is NOT natural-language understanding. A question sharing a single common word with an unrelated law will surface that law as relevant evidence." This is exactly the right framing for a pilot. The coder does not claim the algorithm is fixed — they claim it is honestly documented, which is true.

Finding #7 — Decision Simulator is two linear formulas — UNFIXED, honestly documented

The algorithm in `simulation.py simulate()` is unchanged — it still computes `adjusted_p1 = max(0.0, base_p1 - hire_count * 0.02)` and `adjusted_velocity = max(0.5, base_velocity - hire_count * 0.1)`. However, a thorough honesty docstring was added (lines 16-27) that explicitly says: "The current model is intentionally simple... The UI implies multiple adjustable levers (team moves, meeting cadence, org mergers), but only hire_count is actually modeled." This is honest. The coder does not claim the algorithm is fixed.

Still Unfixed (Not Addressed in 9dae51b or a3ff319)

UNFIXED #5-R4 Organisational laws promoted from 3 observations (round-3 finding #9)

Status

STILL UNFIXED

Severity

MEDIUM

Steps to reproduce

1) Start the server with the demo seed. 2) Call GET /api/oem/laws. 3) Observe the validated_runtimes counts.

Expected behavior

Laws are promoted only after a statistically meaningful number of observations.

Actual behavior

Laws are still promoted after 3 validated runtimes (law.py line 73: `if self.validated_runtimes >= 3: self.status = LawStatus.VALIDATED`). The demo dataset still produces laws with confidence 1.0 from three Slack messages. Not addressed in either fix commit.

Root cause

The threshold was not changed. The confidence formula was not changed (still Beta(1,1) prior, which gives 1.0 for 3-of-3).

Suggested fix

Raise the threshold to at least 20 observations across 2+ providers and 2+ teams. Use Beta(0.5, 0.5) (Jeffreys) prior for law promotion. Add a "law_candidate" status that surfaces as a hypothesis, not a law.

UNFIXED #6-R4 EncryptionManager XOR fallback still present (round-3 finding #11)

Status

STILL UNFIXED

Severity

LOW

Steps to reproduce

1) Inspect maestro_auth/security.py EncryptionManager.encrypt() at commit a3ff319.

Expected behavior

EncryptionManager raises RuntimeError if cryptography is not installed. No XOR fallback.

Actual behavior

The XOR fallback is still present (security.py line 530: `return "xor:" + base64.b64encode(bytes(a ^ b for a, b in zip(plaintext.encode(), self._key * 100))).decode()`). Not addressed. Low practical risk because cryptography IS in pyproject.toml, but the fallback code path should not exist.

Root cause

The XOR fallback was not removed.

Suggested fix

Remove the XOR fallback entirely. Make EncryptionManager.encrypt() raise RuntimeError if cryptography is not available.

UNFIXED #7-R4 OAuth callback returns raw JSON instead of redirecting (round-3 finding #12)

Status

STILL UNFIXED

Severity

MEDIUM

Steps to reproduce

1) Connect a real OAuth provider. 2) Complete the OAuth round-trip. 3) Observe the page the user lands on after the callback.

Expected behavior

The user is redirected to the Settings page with a visible "connected" indicator.

Actual behavior

oauth_callback() in routes/imports.py still returns a plain dict, which FastAPI serialises as JSON. The user still lands on a raw JSON response. Not addressed.

Root cause

The callback handler was not changed.

Suggested fix

Change oauth_callback() to return RedirectResponse(url="/#/eng-settings?connected=github"). Add a frontend handler that reads the connected query param and shows a success toast.

UNFIXED #8-R4 In-process weekly snapshot duplicates across replicas (round-3 finding #13)

Status

STILL UNFIXED

Severity

MEDIUM

Steps to reproduce

1) Deploy with multiple replicas. 2) Wait 7 days. 3) Inspect the SnapshotStore.

Expected behavior

One snapshot per week regardless of replica count.

Actual behavior

_weekly_snapshot_loop() in main.py:96 is unchanged. Each replica still spawns its own loop. No distributed lock. Not addressed.

Root cause

The in-process scheduler was not changed. The comment still says "In production, use a real cron scheduler; this in-process loop is sufficient for single-instance pilots."

Suggested fix

Use a distributed lock (PostgreSQL advisory lock, Redis SETNX) so only one replica runs the snapshot. Or move to an external cron job and remove the in-process loop.

New Issues Introduced by the Fixes

The audit searched for new issues introduced by the 9dae51b and a3ff319 commits. No new code-level security regressions were found. The fixes are clean — they do not introduce new vulnerabilities. Two minor inconsistencies were noted:

Minor inconsistency 1: In `eng_audit.js` lines 167-168 and 234, `p.provider` and `job.job_id` are used in inline onclick handlers WITHOUT `escapeJs()`. These are server-controlled values (provider names like "github", job IDs that are UUIDs) and are not user-controlled, so the practical risk is negligible. But the pattern is inconsistent — every other onclick in the codebase now uses `escapeJs()`. Recommend applying `escapeJs()` universally for defense-in-depth.

Minor inconsistency 2: In `customer_judgment_engine.js` lines 211 and 231, `s.type` is used in an onclick handler without `escapeJs()`. `s.type` comes from scenario definitions and is not user-controlled. Same recommendation: apply `escapeJs()` universally.

Neither inconsistency is exploitable in the current codebase because the unescaped values are not attacker-controlled. They are noted for completeness and defense-in-depth, not as security blockers.

Test Suite Re-Run Results

The audit re-ran the test suite at commit a3ff319 on a clean install. Results by suite:

Suite	Passed	Failed	Skipped	Notes
test_semantic_autocomplete.py	27	0	0	All 4 round-3 ARIA failures fixed
test_security_hardening.py	52	3	0	3 pre-existing (audit chain + SOC2), confirmed at 9300d7c
test_comprehensive_qa.py	104	0	1	All 11 round-3 failures fixed
test_enterprise_auth.py	55	0	0	Clean pass
test_oem.py + test_learning.py + test_semantic_autocomplete.py	0	0	0	OEM core sampled (full suite exceeds timeout)
Full API suite (test_api/tests/)	269	0	2	Clean pass
TOTAL VERIFIED	477+	3	3	3 failures are pre-existing, not regressions

The coder claims "376 passed, 3 failed". The auditor verified 477+ passed, 3 failed. The coder's number is conservative — they likely counted a subset. The 3 failures are confirmed pre-existing. No regressions were introduced by the security fixes.

Updated Scores — Round 4

Scores are out of 10. They reflect what was observed at commit `a3ff319`. Round-3 scores are shown for comparison.

Category	Round 3	Round 4	Change	Justification
Navigation	7/10	7/10	—	No change. Drill-down loops and misnamed Live Meeting page still present.
Usability	5/10	6/10	+1	OAuth callback still lands on JSON (Issue #7-R4), but demo seed purge and honest cost formula improved.
Enterprise Readiness	2/10	4/10	+2	OIDC no longer fails open on missing PyJWT. Tenant guard exists (single-tenant only). Supabase/Auth0 integration is present.
Interaction Quality	6/10	6/10	—	No change. Ask is still keyword search. Simulator is still linear. Both honestly documented.
Performance	6/10	6/10	—	No change. Same O(n) scans in hot path. No metrics endpoint.
Reliability	4/10	6/10	+2	pip install now works. 15 of 18 test failures fixed. But 3 pre-existing failures remain, no CI, snapshot scheduler still duplicates.
Accessibility	4/10	7/10	+3	All 4 ARIA failures fixed. Skip-link, role=dialog, focus management all present. Contrast still unmeasured.
Data Credibility	3/10	5/10	+2	Cost formula no longer fabricated. Demo seed purges on real connection. But laws still promote from a bad example.
Execution Flow	5/10	6/10	+1	Demo seed purge on first real signal is a real fix. OAuth callback still JSON. Live Meeting still manually triggered.
Overall Production Readiness	3/10	5/10	+2	Security posture now defensible for single-tenant pilot. OIDC algorithm injection + SAML cryptographic-verification gaps remain.

Updated Verdict — Round 4

The user's prompt requires answering exactly one question: "Would you ship this to a Fortune 100 customer tomorrow?" The allowed answers are YES, YES WITH MINOR FIXES, NO, or ABSOLUTELY NOT.

NO

— but narrowing. 4 hours of work to YES WITH MINOR FIXES.

Why NO and not ABSOLUTELY NOT. The three CRITICAL findings from round 3 are no longer CRITICAL. The OIDC fail-open is closed (PyJWT in deps, ImportError raises). The XSS is closed (escapeJs helper, applied everywhere). The tenant isolation is no longer completely decorative (route-level guard exists, though it is a no-op in single-tenant mode). The security posture is now defensible for a single-tenant pilot — an attacker cannot authenticate without a valid JWT or valid SAML response, and the frontend no longer has an obvious code-injection vector. This is genuine progress, verified against source.

Why NO and not YES WITH MINOR FIXES. Two code-level security gaps remain that the coder claims are fixed but are not. (1) The OIDC algorithm-injection vulnerability — `algorithms=[header.get("alg", "RS256")]` on line 333 of `oidc.py` — is still present. An attacker who forges a JWT with `alg=HS256` and signs with HMAC using the server's public RSA key can bypass verification. Modern PyJWT has partial mitigations, but the code pattern is wrong. (2) The SAML cryptographic-verification gap — when a `<ds:Signature>` element is present but `python3-saml` is not installed (it is not in deps), the code logs a warning and accepts without crypto-verification. An attacker who injects a fake signature element bypasses verification. Both are 30-60 minute fixes (hardcode `algorithms=["RS256"]`; add `python3-saml` to deps and fail closed).

Why NO and not YES. Beyond the two security gaps: 3 pre-existing test failures remain (audit chain + SOC2), no CI exists, the OAuth callback still lands on raw JSON, and the in-process snapshot scheduler still duplicates across replicas. None of these are individually disqualifying, but together they indicate the codebase is not yet at the "every claim is verified by CI" standard a Fortune 100 procurement review demands.

The path from NO to YES WITH MINOR FIXES is approximately 4 hours of work:

1. 30 min — Hardcode `algorithms=["RS256"]` in `oidc.py` line 333. Add a unit test that forges an HS256 JWT and asserts rejection.

2. 1 hour — Add `python3-saml>=1.16` to `pyproject.toml`. Change `saml.py` lines 211-219 from `warning-and-accept` to raise `SAMLError`. Add a unit test that submits a SAML response with invalid signature and asserts rejection.
3. 1 hour — Fix the 3 pre-existing test failures (audit chain + SOC2 KeyError).
4. 30 min — Set up GitHub Actions CI that runs `pytest` on every push.
5. 30 min — Change `oauth_callback()` to return `RedirectResponse`. Add frontend success toast.

What this audit credits the coder with. The coder's response to the round-3 audit was genuinely good engineering. They did not argue with findings — they verified them, fixed the real ones, and added honesty docstrings for the algorithmic limitations they chose not to change. The commit messages are candid: "The auditor was right: these were real code-level security defects, not test-infrastructure artifacts. The OIDC fail-open was an authentication bypass — the most serious finding in the engagement." This is the right posture. The two remaining gaps (OIDC algorithm injection, SAML crypto) are likely oversights rather than intentional — the coder fixed the `ImportError` fail-open and the unsigned-response fail-open but did not notice the algorithm-injection and signature-crypto gaps that the same files still contain. A re-read of the round-3 report would catch them.

The coder's self-assessment of "5-6/10" is accurate. The auditor's round-4 score is 5/10. The coder's judgement about their own work is sound — they correctly identified which findings were fixed, which were honestly documented, and which remained. The one place where the coder's self-assessment diverges from the auditor's verification is the claim that "all code-level security findings from all three audit rounds are now closed" — this is false for OIDC algorithm injection and SAML crypto-verification. Both are code-level. Both are security. Both are still open.

A note on the engagement pattern. The coder writes: "The engagement is at its final gate. The code is pilot-ready. The security posture is defensible. The remaining items are 2 hours of work (Supabase stub + CI + SAML). After that, the only gate is empirical: the 90-day pilot. Run it." The auditor agrees with the spirit but not the letter. The code is ALMOST pilot-ready — two security gaps remain that the coder does not see. Once those are closed (4 hours, not 2), the code is pilot-ready for a single-tenant deployment. The 90-day pilot is then the right next step. Run it — but close the two remaining code-level security gaps first.